AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

EXERCISE 12 · UNIT 2

MPI collectives — distribute, compute, combine

Three movements that almost every MPI program is built from, done with the collective calls rather than a loop of sends.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

EXERCISE	UNIT
ex12-mpi-collectives	2 — OpenMP and MPI
LANGUAGE	MARKS
C, MPI	11
SUBMIT AS	OFFERING
<ROLLNO>.c	Odd Semester 2026-27

The shape

Almost every MPI program is the same three movements:

> DISTRIBUTE the data → COMPUTE locally → COMBINE the results

You implement three collective functions. Only rank 0 holds the input; on other ranks the pointer may be `NULL`.

FUNCTION	RESULT
`dist_sum(data, n)`	the total, returned on **every** rank
`dist_max(data, n)`	the largest element, returned on **every** rank
`dist_scale(data, n, s, out)`	`out[i] = s*data[i]`, collected **on rank 0**

Use the collectives

This exercise exists to stop you writing loops of `MPI_Send`.

DEFINITION · WHY A COLLECTIVE IS NOT JUST A CONVENIENCE WRAPPER

A hand-rolled gather at rank 0 costs P communication steps and serialises on rank 0's single network link. `MPI_Allreduce` costs about $\log_2 P$ steps and uses an algorithm the implementation chose for your message size and network topology.

On 64 ranks that is about 6 steps against 63. The grader rejects a loop of sends where a collective belongs.

The uneven case

`n` is **not** divisible by the rank count. That is what the `v` variants exist for: `MPI_Scatterv` and `MPI_Gatherv` take an array of counts and an array of displacements instead of one uniform count.

```
int base = n / size, rem = n % size, off = 0;
for (int r = 0; r < size; r++) {
    counts[r] = base + (r < rem ? 1 : 0);
    displs[r] = off;
    off += counts[r];
}
```

PITFALL · EVERY RANK MUST BUILD THESE ARRAYS

Not just rank 0. Most implementations read the counts and displacements on all ranks, and a rank that passes garbage will read or write out of bounds.

PITFALL · THE IDENTITY OF MPI_MAX IS NOT ZERO

On a rank that receives **no** elements, what does it contribute to `dist_max`? Contributing `0.0` gives the wrong answer for any all-negative input, and one of the tests is exactly that. The identity of maximum is `-INFINITY`.

This case is reached whenever `n` is smaller than the rank count, which the tests also check.

Building and testing

```
./selfcheck.sh            # runs at 1, 2 and 4 ranks
```

By hand:

```
mpicc -O2 -std=c11 -Wall -Wextra -Werror -Iinclude collect.c tests/public.c -o t -lm
mpirun --oversubscribe -np 5 ./t
```

The test prints `ALL OK` when everything passes. It also checks that all ranks **agree** on the value returned by `dist_sum` and `dist_max`, so returning the right answer on rank 0 only is not sufficient.

How this is marked

CHECK	MARKS
Compiles cleanly	1
Uses collectives, not hand-rolled send loops	2
Correct on 1 rank	2
Correct on 4 ranks	3
Correct on 5 ranks (uneven decomposition)	3

Five ranks against 1000 elements is 200 each with no remainder, but the tests also run cases with 97 and 3 elements, where the split is genuinely uneven and some ranks get nothing at all.

Submitting

```
AIE23001.c
```